R-Übungsaufgaben zu MfW7 - Multivariate Optimierung

Aufgaben und Kontrollergebnis

Hinweis: Beachten Sie, dass sämtliche Aufgaben auf den R Übungsblättern auch via R gelöst werden müssen, sofern nicht anders gekennzeichnet! Beachten Sie ferner, dass die Aufgabenmenge keinen Hinweis auf die Relevanz des Themas darstellt. Die hier vergleichsweise geringe Anzahl an Aufgaben ist darauf zurückzuführen, dass es sich insgesamt um zeitintensivere Aufgaben handelt.

R 7.1 - Numerische Optimierung in R

Aufgabe 1.
Bestimmen Sie das Minimum der Funktion $h(a,b) = e^{a^2} + b^2$. Berechnen Sie zunächst einige Funktionswerte, um einen geeigneten Startwert zu finden.

```
#Funktionsobjekt definieren
> h <- function(vec) {
+   a <- vec[1]
+   b <- vec[2]
+   return(exp(a^2)+b^2)
+ }

> nlm(h,c(2,4))
$minimum
[1] 1

$estimate
[1] -5.013329e-07 -4.995815e-07

$gradient
[1] -2.664535e-09  8.881784e-10

$code
[1] 1

$iterations
[1] 7
#Hinweis: Beachten Sie die Fehlermeldung. Diese koennen Sie jedoch ignorieren. Sie wird durch die gewaehlten
      Startwerte verursacht.
```

Aufgabe 2.
Bestimmen Sie die Extremstellen der Funktion $f(x,y) = -x^2 - y^2 + 10x + 20y$

Hinweis: *Wenn der Funktionsverlauf bekannt ist, ist Transformation nicht zwingend notwendig!*

```
#Funktionsobjekt definieren
> f <- function(vec) {
+   x <- vec[1]
+   y <- vec[2]
+   return(-(x^2)-(y^2)+10*x+20*y)
+ }

#Maximum
#Hinweis: Wichtig, die Funktion muss transformiert werden, da der Befehl nlm grds. minimiert!
> f_neg <- function(vec) {
+   x <- vec[1]
+   y <- vec[2]
+   return((x^2)+(y^2)-10*x-20*y)
+ }
> nlm(f_neg, c(1,5))
$minimum
[1] -125 #Hier - aufgrund der Transformation - negativer Wert!

$estimate
[1] 5.000001 9.999999

$gradient
[1] 6.829736e-06 8.537882e-06

$code
[1] 1

$iterations
[1] 1

# Minimum
```

```
> nlm(f, c(1,5))
$minimum
[1] -909804488 #zeigt bereits, dass nicht konvergiert. Folglich liegt auch kein lokales Minimum vor.

$estimate
[1] -18837.66 -23543.32

$gradient
[1] 37685.30 47106.62 #Hinweis: Gradient sollte nahe 0 sein.

$code
[1] 5 #Hinweis: Angestrebt ist Code = 1. Dies koennte eine Begruendung darstellen, dass es sich hier um kein
      Minimum handelt. Beachten Sie ansonsten den Funktionswert und die Funktion um zu beurteilen, ob das
     Ergebnis plausibel ist.

$iterations
[1] 11
```

Aufgabe 3.
Untersuchen Sie, ob globale Extremstellen der Funktion $g(x, y, z) = 3x^2 + 0.5y^2 - z^2 - 12x$ existieren.

```
> g <- function(vec) {
+   x <- vec[1]
+   y <- vec[2]
+   z <- vec[3]
+   return(3*(x^2)+0.5*(y^2)-(z^2)-12*x)
+ }

#Maximum (Transformation notwendig, da nlm() grds. minimiert.
> g_neg <- function(vec) {
+   x <- vec[1]
+   y <- vec[2]
+   z <- vec[3]
+   return(-3*(x^2)-0.5*(y^2)+(z^2)+12*x)
+ }
> nlm(g_neg, c(1,0))
$minimum
[1] 1.797693e+308 #zeigt bereits, dass keine globale Maximumstelle verfuegbar

$estimate
[1] 1 0

$gradient
[1] 0 0

$code
[1] 1

$iterations
[1] 0

#Minimum
> nlm(g,c(1,0))
$minimum
[1] 1.797693e+308 #zeigt bereits, dass keine globale Minimumstelle verfuegbar

$estimate
[1] 1 0

$gradient
[1] 0 0

$code
[1] 1

$iterations
[1] 0
```

R 7.2 - (Augmentierte) Lagrange-Methode in R

Aufgabe 4.
Bestimmen Sie das Minimum der Funktion $i(x, y) = x^2 + y^2$ unter der Nebenbedingung $x + y = 4$.

```
# Notwendige Pakete installieren
> install.packages("Rsolnp")
> library(Rsolnp)

#Funktionsobjekt definieren
> i <- function(vec) {
+   x <- vec[1]
+   y <- vec[2]
+   return(x^2 + y^2)
+ }

#Nebenbedingung definieren
> NB <- function(vec) {
+   x <- vec[1]
+   y <- vec[2]
+   return(x+y)
+ }
> solnp(c(1,2),fun = i, eqfun = NB, eqB = 4)

Iter: 1 fn: 8.0000   Pars:  2.00000 2.00000
Iter: 2 fn: 8.0000   Pars:  2.00000 2.00000
solnp--> Completed in 2 iterations
$pars
[1] 2 2

$convergence
[1] 0

$values
[1] 5 8 8

$lagrange
     [,1]
[1,]    4

$hessian
     [,1] [,2]
[1,]  1.5 -0.5
[2,] -0.5  1.5

$ineqx0
NULL

$nfuneval
[1] 25

$outer.iter
[1] 2

$elapsed
Time difference of 0.08701682 secs

$vscale
[1] 8e+00 1e-08 2e+00 2e+00
```

Aufgabe 5.
Bestimmen Sie das Maximum der Funktion $j(x, y, z) = -x^2 + y - z$ unter den Nebenbedingungen $x + y = 0$ und $x - z = -2$.

```
# Notwendige Pakete installieren
> install.packages("Rsolnp")
> library(Rsolnp)

#Funktionsobjekt definieren
> j <- function(vec){
+   x<-vec[1]
+   y<-vec[2]
+   z<-vec[3]
+   return(-(-(x^2)+y-z)) #Hier erfolgt ebenfalls bereits die negative Transformation der Funktion, da hier
     eine Maximierung durchgefuehrt wird (vgl. Befehl nlm())
+ }
> NB2 <- function(vec){
+   x<-vec[1]
+   y<-vec[2]
```

```
+   z<-vec[3]
+   t = x+y
+   r = x-z
+   return(c(t,r))
+ }
> solnp(c(1,2,3),fun = j, eqfun = NB2, eqB = c(0,-2))

Iter: 1 fn: 1.0000   Pars:  -1.00000  1.00000  1.00000
Iter: 2 fn: 1.0000   Pars:  -1.00000  1.00000  1.00000
solnp--> Completed in 2 iterations
$pars
[1] -1.0000001  1.0000001  0.9999999

$convergence
[1] 0

$values
[1] 2 1 1

$lagrange
     [,1]
[1,]   -1
[2,]   -1

$hessian
           [,1]      [,2]       [,3]
[1,]  2.6666667 0.3333333 -0.3333333
[2,]  0.3333333 0.6666667  0.3333333
[3,] -0.3333333 0.3333333  0.6666667

$ineqx0
NULL

$nfuneval
[1] 40

$outer.iter
[1] 2

$elapsed
Time difference of 0.05983615 secs

$vscale
[1] 0.99999999 0.00000001 0.00000001 1.00000005 1.00000006 0.99999995
```